

SI4006 · Entrega M1 — Fine-tuning baseline del proyecto

Módulo 1 · Arquitectura transformer y fine-tuning eficiente **Peso:** 10% de la nota final · **Modalidad:** equipo · **Entrega:** Sesión 4 (fin de clase)

Brief

Tomen el modelo base que eligieron según lo visto en la clase del 31 de Julio, afínenlo (fine-tuning) con LoRA (idealmente, o con algo igual de eficiente o mejor) sobre texto real de su dominio para que resuelva la tarea central de su proyecto, y muestren con números que quedó mejor que un punto de partida razonable.

Por qué esta entrega

Todo su proyecto integrador arranca de un modelo que entienda su dominio. Ya "decidieron" **qué familia** (encoder, decoder o encoder-decoder) y **qué modelo base** encaja con su tarea. La idea de esta siguiente semana (4), es que aprendan a usar el ecosistema Hugging Face, la Trainer API y LoRA. Ahora, van a convertir esa decisión en un modelo entrenado y **medido**. Lo importante de esta entrega no es lograr el mejor número posible, sino **tomar decisiones técnicas justificadas y compararlas honestamente, y razonablemente, contra un baseline**.

Esto no se descarta luego, se acumula. El modelo que entreguen aquí es el que probablemente terminarán evaluarando con rigor en el tema que sigue, sobre el que montarán RAG y al que sumarán un componente visual.

Qué deben entregar

Un **repositorio de GitHub** con estos elementos:

1. Notebook de fine-tuning ejecutable (Colab)

Un notebook que corra de principio a fin en Colab gratuito (GPU T4) e incluya, claramente separado en secciones:

- Carga del modelo base y del tokenizer desde el Hub.
- Carga y preparación del dataset del dominio (limpieza mínima, split train/validation). Probablemente, dado que sus proyectos son bastante específico, tendrán que hacer un proceso separado para la recolección de datos y su adecuación. Si es el caso, deben incluir en una subcarpeta el proceso de recolección y pre-procesamiento.
- Configuración de LoRA (**rank**, **alpha**, **target_modules**), o el "acelerador" de entrenamiento que elijan, y del entrenamiento (Trainer API).
- El entrenamiento en sí, con la métrica evaluada durante el proceso (NO borren los outputs antes de enviar, solo eviten sobre printear para que no pese una tonelada).
- La evaluación final sobre el conjunto de validación, **comparada contra el baseline** (ver más abajo).

- Al menos **3 ejemplos cualitativos** de entrada → salida del modelo afinado (que se vea qué hace).

2. Descripción del dataset (1–2 páginas, en el README o un **.md** aparte)

- **Fuente:** de dónde salió el texto (URL, dataset abierto, scraping autorizado, datos propios).
- **Tamaño:** cuántos ejemplos, y el split que usaron (train / validation).
- **Idioma(s) y licencia** (¿hay algo que restrinja el uso?).
- **Tarea:** qué es el input y qué es el output esperado (clasificación / extracción / resumen / QA / generación / transformación).
- **Sesgos o limitaciones conocidas** del dataset: qué le falta, en qué casos probablemente falle.

3. Reporte de resultados (suscinto, en el README)

- **Modelo base y familia elegidos**, con **una frase que justifique por qué esa familia** para su tarea (enganchen con lo de la Semana 3: objetivo de pre-entrenamiento ↔ tarea).
- **Baseline con el que comparan** (ver la sección siguiente) y **por qué ese**.
- **Tabla de resultados:** métrica principal del baseline vs. métrica del modelo afinado, sobre el mismo conjunto de validación.
- **Una frase de lectura honesta:** ¿mejoró? ¿cuánto? ¿en qué casos sigue fallando?

Especificaciones técnicas (lo no negociable)

- **Fine-tuning acelerado.** No full fine-tuning: el punto del módulo es la adaptación eficiente y que corra en Colab gratis. Deben poder nombrar sus hiperparámetros clave (**rank**, **alpha**, **target_modules**) y decir por qué los eligieron (la parte divertida, experimenten).
- **Comparación contra un baseline obligatoria.** Un número solo no dice nada. El baseline puede ser: clase mayoritaria, una regla/regex, o —lo más recomendable— **el mismo modelo base SIN fine-tuning** (zero-shot). La gracia es mostrar el *delta* que aportó el fine-tuning.
- **Métrica adecuada a la tarea.** Clasificación → accuracy / F1. Generación o resumen → ROUGE, u otra justificada. Elijan una principal y digan por qué es la correcta para su caso (esto se profundiza en M2, aquí basta con una elección razonable). Como les he mencionado, no tiene que ser la métrica ideal ahora, pero sí debe tener sentido.
- **Reproducibilidad.** Que otra persona pueda abrir el notebook y correrlo. Fijen una semilla, dejen las celdas en orden, no dejen rutas locales rotas.
- **Debe correr en Colab gratuito.** Si no cabe, el modelo base es demasiado grande: bajen de tamaño.

Rúbrica (100 puntos = el 10%)

Criterio	Puntos	Qué buscamos
Fine-tuning funcional con LoRA	30	El notebook corre end-to-end en Colab; LoRA bien configurado; el modelo efectivamente se entrena.
Baseline y comparación	25	Hay un baseline explícito y justificado; la comparación es sobre el mismo conjunto y honesta.

Criterio	Puntos	Qué buscamos
Descripción del dataset	20	Fuente, tamaño, idioma, licencia, tarea y sesgos, todo claro y verificable. Si deben hacer procesamiento de información, los scripts o notebooks de procesamientos entran acá, y dependiendo de la complejidad añadida de su dataset, podrá corresponder hasta un 10% adicional de la nota (puntos extra).
Justificación de las decisiones	15	Por qué esa familia, ese modelo base y esos hiperparámetros; conexión con la tarea del proyecto.
Reproducibilidad y orden	10	Semilla fijada, celdas en orden, repositorio legible, ejemplos cualitativos presentes.

Semilla de ética (se retoma con más peso en el proyecto final): que la descripción del dataset nombre al menos un sesgo o limitación es parte del criterio de dataset — no es opcional.

Formato y entrega

- **Un enlace por equipo** repositorio GitHub público subido a **EAFIT Interactiva** antes del 14 de Agosto de 2026.
- El README debe abrir con **una frase en lenguaje sencillo** que diga qué hace su sistema y para quién — alguien ajeno al equipo debe entenderlo sin leer código.
- Nombren el repo con los nombres de los miembros del equipo.
- Un solo integrante hace la entrega, pero **todos deben poder explicar cualquier parte** (esto se verifica en las siguientes semanas y en la sustentación final).

Política de entregas tardías: cada día calendario de retraso descuenta 2% del peso de la entrega, hasta un máximo de 7 días (política general del curso).

De dónde sacan cada cosa (ya la tienen)

HuggingFace es la fuente más fácil, tanto para modelo, datos, tutoriales de entrenamiento, pero no es obligatorio que sea este.

Consejo del Lab A (Semana 3): antes de casarse con un modelo base, miren **cómo tokeniza el vocabulario de su dominio**. Un modelo que parte sus términos clave en muchos tokens rinde peor y cuesta más. Es un argumento técnico que casi nadie trae — y aquí suma en "justificación de las decisiones".

Checklist antes de entregar

- ☐ El notebook corre de principio a fin en Colab gratuito, sin errores.
- ☐ El notebook/script del procesamiento de datos, de ser necesarios.
- ☐ Usamos LoRA (no full fine-tuning) y sabemos explicar **rank**, **alpha** y **target_modules**.
- ☐ Hay un baseline explícito y comparamos contra él sobre el mismo conjunto de validación.
- ☐ La descripción del dataset incluye fuente, tamaño, idioma, licencia y al menos un sesgo.

- ☐ El README abre con una frase entendible por alguien sin formación en IA.
- ☐ Justificamos por qué esa familia y ese modelo base para nuestra tarea.
- ☐ Fijamos semilla y dejamos el repositorio ordenado y reproducible.
- ☐ Incluimos al menos 3 ejemplos cualitativos de entrada → salida.